SZABIST
University

Shaheed Zulfikar Ali Bhutto
Institute of Science and Technology

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks: _____

Obtained Marks: _____

Computing Vision

Lab Task # 15

Submitted To: **Mr. Muhammad Azhar**

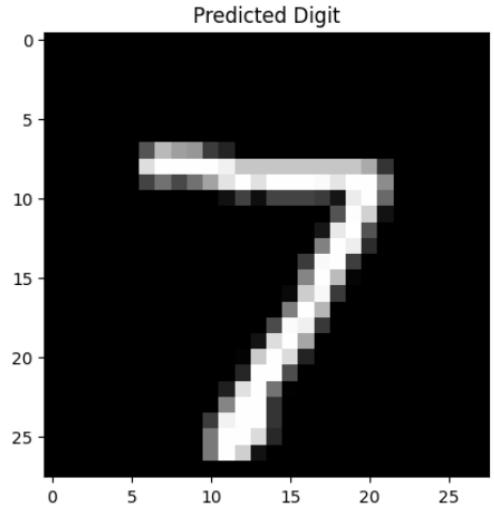
Student Name: **Ayesha Munir**

Reg. Number: **23108106**

Code Example 1: Load and Preprocess MNIST Data

```python
# Code Example 1: Load and Preprocess MNIST Data
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(2, 2),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(2, 2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, validation_split=0.1)

loss, accuracy = model.evaluate(x_test, y_test)

print(f"Test Accuracy: {accuracy * 100:.2f}%")

plt.imshow(x_test[0].reshape(28, 28), cmap='gray')
plt.title("Predicted Digit")
plt.show()

pred = model.predict(x_test[0].reshape(1, 28, 28, 1))

print("Predicted Label:", pred.argmax())
```

```
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 14s 8ms/step - accuracy: 0.9572 - loss: 0.1372 - val_accuracy: 0.9860 - val_loss: 0.0467
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 14s 8ms/step - accuracy: 0.9859 - loss: 0.0454 - val_accuracy: 0.9863 - val_loss: 0.0508
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 14s 8ms/step - accuracy: 0.9897 - loss: 0.0313 - val_accuracy: 0.9903 - val_loss: 0.0353
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 14s 8ms/step - accuracy: 0.9927 - loss: 0.0227 - val_accuracy: 0.9915 - val_loss: 0.0333
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 14s 8ms/step - accuracy: 0.9943 - loss: 0.0178 - val_accuracy: 0.9917 - val_loss: 0.0332
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9904 - loss: 0.0306
Test Accuracy: 99.04%
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 108ms/step
Predicted Label: 7
```

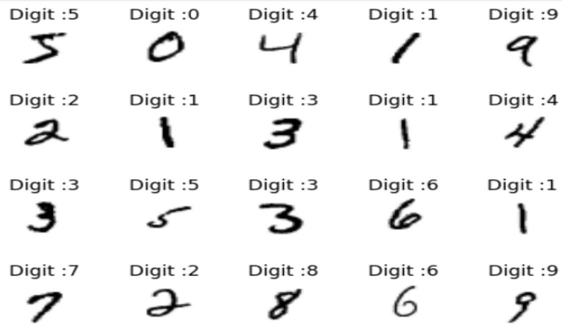
Code Example 2: Handwritten Digit Recognition Using Neural Networks (Keras & TensorFlow)

A fintech startup is building a mobile banking application that supports remote cheque deposit and digital form filling. One major challenge is accurately reading handwritten digits on scanned forms—such as account numbers, routing codes, and amounts—written by users with varying handwriting styles and image quality.

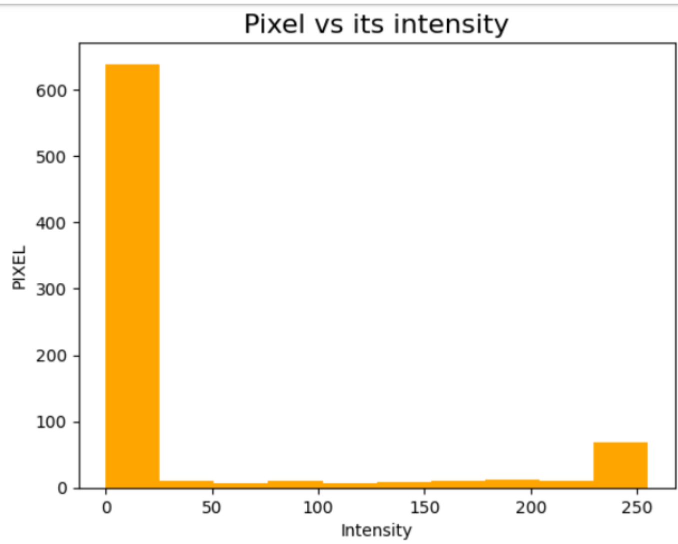
Traditional OCR systems perform poorly with unconstrained handwriting due to noise, distortion, and inconsistent writing. The company decides to implement a deep learning-based digit recognition model to solve this problem with high accuracy and robustness. They choose Keras with TensorFlow backend for its simplicity, flexibility, and strong support for neural network development.

```
# Code Example 2: Handwritten Digit Recognition Using Neural Networks (Keras & TensorFlow)
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
from keras.datasets import mnist

objects = mnist

(train_img, train_lab), (test_img, test_lab) = objects.load_data()

for i in range(20):
    plt.subplot(4, 5, i + 1)
    plt.imshow(train_img[i], cmap='gray_r')
    plt.title("Digit :{}".format(train_lab[i]))
    plt.subplots_adjust(hspace=1)
    plt.axis('off')

plt.show()

print('Training images shape :', train_img.shape)
print('Testing images shape :', test_img.shape)

print("How image looks like :")
print(train_img[0])

plt.hist(train_img[0].reshape(784), facecolor='orange')
plt.title('Pixel vs its intensity', fontsize=16)
plt.ylabel('PIXEL')
plt.xlabel('Intensity')
plt.show()

train_img = train_img / 255.0
test_img = test_img / 255.0

print('How image looks like after normalising: ')
print(train_img[0])

plt.hist(train_img[0].reshape(784), facecolor='orange')
plt.title('Pixel vs its intensity', fontsize=16)
plt.ylabel('PIXEL')
plt.xlabel('Intensity')
plt.show()
```

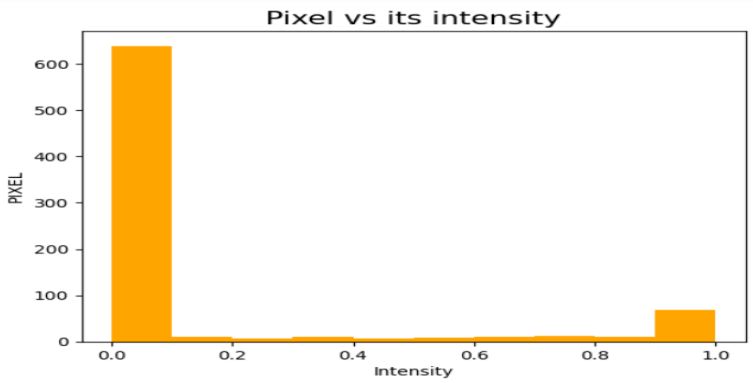
```
from keras.models import Sequential
from keras.layers import Flatten, Dense

model = Sequential()

input_layer = Flatten(input_shape=(28, 28))
model.add(input_layer)

hidden_layer1 = Dense(512, activation='relu')
model.add(hidden_layer1)

hidden_layer2 = Dense(512, activation='relu')
model.add(hidden_layer2)

output_layer = Dense(10, activation='softmax')
model.add(output_layer)

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(train_img, train_lab, epochs=5)

model.save('project.h5')
```




```
Training images shape : (60000, 28, 28)
Testing images shape : (10000, 28, 28)
How image looks like :
[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   3  18  18  18 126 136
  175  26 166 255 247 127   0   0   0   0]
 [  0   0   0   0   0   0   0   0  30  36  94 154 170 253 253 253 253 253
  225 172 253 242 195  64   0   0   0   0]
 [  0   0   0   0   0   0   0  49 238 253 253 253 253 253 253 253 253 251
   93  82  82  56  39   0   0   0   0   0]
 [  0   0   0   0   0   0   0  18 219 253 253 253 253 253 198 182 247 241
    0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0  80 156 107 253 253 205  11   0  43 154
```




```
How image looks like after normalising:
[[0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.        ]
 [0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.        ]
 [0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.        ]
 [0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
```




```
Epoch 1/5
C:\Users\dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer
  super().__init__(**kwargs)
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.9430 - loss: 0.1825
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9747 - loss: 0.0805
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9814 - loss: 0.0569
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9868 - loss: 0.0418
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9893 - loss: 0.0347
```

Case Study : Postal Code Recognition in Postal Systems

A national postal service wants to automate the sorting of mail by recognizing handwritten zip codes on envelopes. Manual sorting is time-consuming and prone to error, especially during peak mailing seasons.

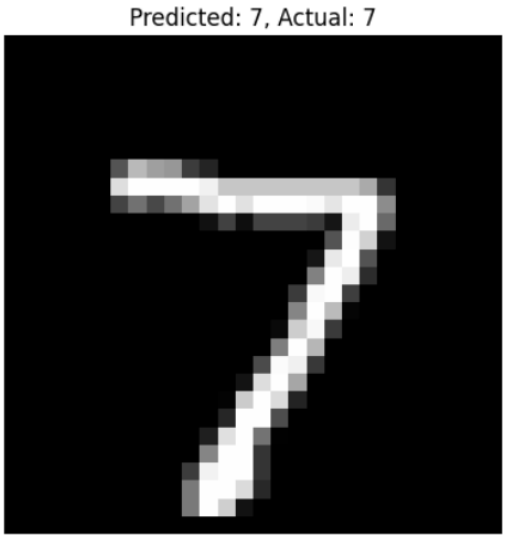
Objective:

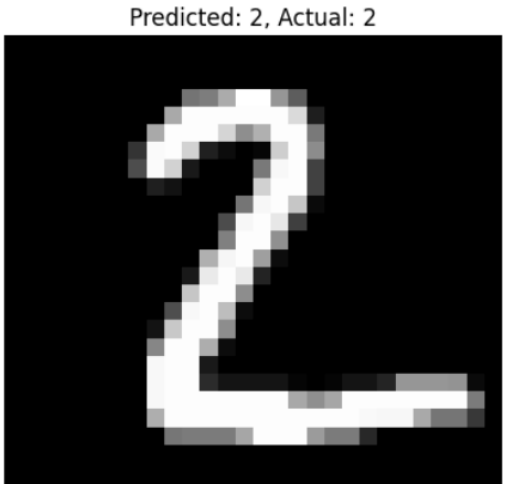
Implement a neural network that can recognize handwritten digits (0–9) from scanned postal codes to enable automatic mail categorization and routing.

```python
# Case Study : Postal Code Recognition in Postal Systems
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_data=(x_test, y_test)
)

test_loss, test_acc = model.evaluate(x_test, y_test)

print(f"\nTest Accuracy: {test_acc:.4f}")

predictions = model.predict(x_test)
```

```python
predictions = model.predict(x_test)

for i in range(5):
    plt.imshow(x_test[i], cmap='gray')
    plt.title(
        f"Predicted: {np.argmax(predictions[i])}, Actual: {np.argmax(y_test[i])}"
    )
    plt.axis('off')
    plt.show()
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.9312 - loss: 0.2397 - val_accuracy: 0.9658 - val_loss: 0.1153
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9694 - loss: 0.1027 - val_accuracy: 0.9696 - val_loss: 0.0950
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9775 - loss: 0.0713 - val_accuracy: 0.9756 - val_loss: 0.0803
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9826 - loss: 0.0546 - val_accuracy: 0.9761 - val_loss: 0.0770
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9864 - loss: 0.0419 - val_accuracy: 0.9728 - val_loss: 0.0887
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9728 - loss: 0.0887

Test Accuracy: 0.9728
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```





Case Study 1: Cheque Processing in Banking

Scenario:

A bank processes thousands of handwritten cheques daily. Reading account numbers and cheque amounts written in digits manually is inefficient and error-prone.

Objective:

Use digit recognition to automate the reading of handwritten cheque fields, reducing transaction processing time and increasing accuracy.

```python
# Case Study 1: Cheque Processing in Banking
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D())

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D())

model.add(Flatten())

model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(x_train, y_train, epochs=5)

loss, accuracy = model.evaluate(x_test, y_test)

print("Accuracy:", accuracy)

model.save("Cheque_Digit_Recognition.h5")
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.9622 - loss: 0.1226
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.9872 - loss: 0.0410
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 15s 8ms/step - accuracy: 0.9911 - loss: 0.0280
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 15s 8ms/step - accuracy: 0.9939 - loss: 0.0191
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 15s 8ms/step - accuracy: 0.9956 - loss: 0.0141
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9923 - loss: 0.0246
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
recommend using instead the native Keras format, e.g. `model.save('my_model.keras')`
Accuracy: 0.9922999739646912
```

Case Study 2: Exam Paper Evaluation System

Scenario:

An edtech company develops an automated evaluation platform that scans student answer sheets and reads handwritten numerical answers. Teachers want quick and consistent grading.

Objective:

Train a digit recognition model to interpret students' handwritten numerical answers and assist in automated scoring of digit-based questions.

```
# Case Study 2: Exam Paper Evaluation System
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential()

model.add(Flatten(input_shape=(28,28)))

model.add(Dense(128, activation='relu'))
model.add(Dense(64, activation='relu'))

model.add(Dense(10, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(x_train, y_train, epochs=5)

loss, accuracy = model.evaluate(x_test, y_test)

print("Accuracy:", accuracy)

predictions = model.predict(x_test)

print("Predicted Answer:", predictions[0].argmax())

model.save("Exam_Evaluation_Model.h5")
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9290 - loss: 0.2422
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9682 - loss: 0.1045
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9774 - loss: 0.0720
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9828 - loss: 0.0552
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9853 - loss: 0.0447
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9767 - loss: 0.0838
Accuracy: 0.9767000079154968
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Predicted Answer:
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.
recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
```

Case Study 3: Multilingual Digit Recognition in International Parcel Services

Scenario:

A global parcel delivery company receives shipments from around the world. Many of these packages have handwritten numbers (e.g., postal codes, tracking numbers) in various numeral styles—Arabic, Devanagari, Latin, etc.

Traditional models trained only on Western digits (0–9) fail to generalize across numeral systems, leading to errors in sorting and delays in delivery.

Objective:

Develop a multilingual digit recognition system using deep learning that can accurately recognize handwritten numerals across multiple writing systems. The model must be trained on datasets containing diverse numeral styles and must generalize across regional handwriting variations.

```
# Case Study 3: Multilingual Digit Recognition
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D())

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D())

model.add(Flatten())

model.add(Dense(256, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(x_train, y_train, epochs=5, validation_split=0.2)

loss, accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", accuracy)

model.save("Multilingual_Digit_Model_MNIST.h5")
```

```
Epoch 1/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 9ms/step - accuracy: 0.9586 - loss: 0.1350 - val_accuracy: 0.9818 - val_loss: 0.0612
Epoch 2/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 14s 10ms/step - accuracy: 0.9862 - loss: 0.0449 - val_accuracy: 0.9862 - val_loss: 0.0427
Epoch 3/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9905 - loss: 0.0300 - val_accuracy: 0.9869 - val_loss: 0.0447
Epoch 4/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9934 - loss: 0.0209 - val_accuracy: 0.9890 - val_loss: 0.0399
Epoch 5/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 14s 10ms/step - accuracy: 0.9947 - loss: 0.0164 - val_accuracy: 0.9881 - val_loss: 0.0480
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9879 - loss: 0.0387
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file fo
recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my
Test Accuracy: 0.9879000186920166
```

Case Study 4: Offline Digit Recognition on Edge Devices for Rural Banking

Scenario:

A rural banking initiative deploys low-power handheld devices to remote areas for collecting handwritten deposit and withdrawal forms. Since internet access is limited, the digit recognition system must run offline on these edge devices with minimal computational resources.

Objective:

Build and optimize a lightweight digit recognition model using TensorFlow Lite that can run efficiently on edge hardware (such as Raspberry Pi or ARM-based devices). The system should maintain high accuracy while using minimal memory and processing power.

```python
# Case Study 4: Offline Digit Recognition on Edge Devices
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = Sequential()

model.add(Flatten(input_shape=(28,28)))

model.add(Dense(64, activation='relu'))

model.add(Dense(10, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(x_train, y_train, epochs=5)

loss, accuracy = model.evaluate(x_test, y_test)

print("Accuracy:", accuracy)

converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()

with open("Digit_Model.tflite", "wb") as f:
    f.write(tflite_model)
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9151 - loss: 0.3043
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9554 - loss: 0.1530
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9675 - loss: 0.1109
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9743 - loss: 0.0859
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9788 - loss: 0.0701
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9696 - loss: 0.0997
Accuracy: 0.9696000218391418
INFO:tensorflow:Assets written to: C:\Users\dell\AppData\Local\Temp\tmptvg3ts0d\assets
INFO:tensorflow:Assets written to: C:\Users\dell\AppData\Local\Temp\tmptvg3ts0d\assets
Saved artifact at 'C:\Users\dell\AppData\Local\Temp\tmptvg3ts0d'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.float32, name='keras_tensor_93')
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  2466303248720: TensorSpec(shape=(), dtype=tf.resource, name=None)
  2466303259472: TensorSpec(shape=(), dtype=tf.resource, name=None)
  2466303251792: TensorSpec(shape=(), dtype=tf.resource, name=None)
  2466303257744: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

Case Study 5: Handwritten Digit Verification for Fraud Detection in Financial Forms
Scenario:
A financial institution processes loan and insurance forms filled manually by customers. Occasionally, tampered or forged forms are submitted with altered digits—like income amounts or account numbers—written in different handwriting styles.
The organization wants to add an automated verification step that flags inconsistent or suspicious digits based on learned handwriting profiles.
Objective:
Use neural networks to recognize not only the digit class but also learn handwriting styles. Compare digits written by the same person on a single form to detect anomalies. This involves combining digit classification with writer-style clustering or signature profiling.

```
# Case Study 5: Handwritten Digit Verification for Fraud Detection
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical
from sklearn.cluster import KMeans
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

input_layer = Input(shape=(28, 28, 1))

x = Conv2D(32, (3, 3), activation='relu')(input_layer)
x = Flatten()(x)

feature_layer = Dense(128, activation='relu')(x)
output_layer = Dense(10, activation='softmax')(feature_layer)

model = Model(inputs=input_layer, outputs=output_layer)

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(x_train, y_train, epochs=5, validation_split=0.1)

loss, accuracy = model.evaluate(x_test, y_test)
print("Accuracy:", accuracy)

feature_model = Model(
    inputs=model.input,
    outputs=feature_layer
)
```

```
features = feature_model.predict(x_test[:100])

kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(features)

print("Cluster Labels:", clusters[:20])
sim = cosine_similarity(
    features[0].reshape(1, -1),
    features[1].reshape(1, -1)
)

print("Similarity Score:", sim[0][0])

if sim[0][0] < 0.5:
    print("Potential Fraud Detected (Different Writing Style)")
else:
    print("Writing Style Consistent")

model.save("Fraud_Digit_Verification_Model.h5")
```

```
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 37s 22ms/step - accuracy: 0.9564 - loss: 0.1416 - val_accuracy: 0.9840 - val_loss: 0.0629
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 36s 21ms/step - accuracy: 0.9867 - loss: 0.0424 - val_accuracy: 0.9843 - val_loss: 0.0617
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 36s 21ms/step - accuracy: 0.9924 - loss: 0.0234 - val_accuracy: 0.9838 - val_loss: 0.0603
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 36s 21ms/step - accuracy: 0.9953 - loss: 0.0137 - val_accuracy: 0.9823 - val_loss: 0.0715
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 36s 21ms/step - accuracy: 0.9967 - loss: 0.0098 - val_accuracy: 0.9840 - val_loss: 0.0705
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9829 - loss: 0.0644
Accuracy: 0.9829000234603882
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_
Cluster Labels: [0 2 0 2 0 0 0 0 2 0 2 2 0 2 0 1 0 0 1 0]
Similarity Score: 0.32106793
Potential Fraud Detected (Different Writing Style)
```

https://github.com/Ayesha-Munir-Chohan/